

part I. Generics

All the type syntaxes you've learned about so far are meant to be used with types that are completely known when they're being written. Sometimes, however, a piece of code may be intended to work with various different types depending on how it's called.

Take this `identity` function in JavaScript meant to receive an input of any possible type and return that same input as output. How would you describe its parameter type and return type?

```
function identity(input) {  
    return input;  
}  
  
identity("abc");  
identity(123);  
identity({ quote: "I think your self emerges more clearly over time." });
```

We could declare `input` as `any`, but then the return type of the function would also be `any`:

```
function identity(input: any) {  
    return input;  
}  
  
let value = identity(42); // Type of value: any
```

Given that `input` is allowed to be any input, we need a way to say that there is a relationship between the `input` type and the type the function returns. TypeScript captures relationships between types using *generics*.

In TypeScript, constructs such as functions may declare any number of generic *type parameters*: types that are determined for each usage of the generic construct. These type parameters are used as types in the construct to represent some type that can be different in each instance of the construct. Type parameters may be provided with different

types, referred to as *type arguments*, for each instance of the construct but will remain consistent within that instance.

Type parameters typically have single-letter names like `T` and `U` or PascalCase names like `Key` and `Value`. In all of the constructs covered in this part, generics may be declared using `<` and `>` brackets, like `someFunction<T>` or `SomeInterface<T>`.

Generic Functions

A function may be made generic by placing an alias for a type parameter, wrapped in angle brackets, immediately before the parameters parentheses. That type parameter will then be available for usage in parameter type annotations, return type annotations, and type annotations inside the function's body.

The following version of `identity` declares a type parameter `T` for its `input` parameter, which allows TypeScript to infer that the return type of the function is `T`. TypeScript can then infer a different type for `T` every time `identity` is called:

```
function identity<T>(input: T) {  
    return input;  
}  
  
const myString = identity("me"); // Type: "me"  
const myNumber = identity(123); // Type: 123
```

Arrow functions can be generic too. Their generic declarations are also placed immediately before the `(` before their list of parameters.

The following arrow function is functionally the same as the previous declaration:

```
const identity = <T>(input: T) => input;  
  
identity(123); // Type: 123
```

Adding type parameters to functions in this way allows them to be reused with different inputs while still maintaining

type safety and avoiding `any` types.

Explicit Generic Call Types

Most of the time when calling generic functions, TypeScript will be able to infer type arguments based on how the function is being called. For example, in the previous examples' `identity` functions, TypeScript's type checker used an argument provided to `identity` to infer the corresponding function parameter's type argument.

Unfortunately, as with class members and variable types, sometimes there isn't enough information from a function's call to inform TypeScript what its type argument should resolve to. This will commonly happen if a generic construct is provided another generic construct whose type arguments aren't known.

TypeScript will default to assuming the `unknown` type for any type argument it cannot infer.

For example, the following `logWrapper` function takes in a callback with a parameter type set to `logWrapper`'s type parameter `Input`. TypeScript can infer the type argument if `logWrapper` is called with a callback that explicitly declares its parameter type. If the parameter type is implicit, however, TypeScript has no way of knowing what `Input` should be:

```
function logWrapper<Input>(callback: (input: Input) => void) {
  return (input: Input) => {
    console.log("Input:", input);
    callback(input);
  };
}

// Type: (input: string) => void
logWrapper((input: string) => {
  console.log(input.length);
});

// Type: (input: unknown) => void
logWrapper((input) => {
  console.log(input.length);
  //           ~~~~~~
```

```
// Error: Property 'length' does not exist on type 'unknown'.
});
```

To avoid defaulting to `unknown`, functions may be called with an explicit generic type argument that explicitly tells TypeScript what that type argument should be instead. TypeScript will perform type checking on the generic call to make sure the parameter being requested matches up to what's provided as a type argument.

Here, the `logWrapper` seen previously is provided with an explicit `string` for its `Input` generic. TypeScript can then infer that the callback's `input` parameter of generic type `Input` resolves to type `string`:

```
// Type: (input: string) => void
logWrapper<string>((input) => {
  console.log(input.length);
});

logWrapper<string>((input: boolean) => {
  // ~~~~~
  // Argument of type '(input: boolean) => void' is not
  // assignable to parameter of type '(input: string) => void'.
  //   Types of parameters 'input' and 'input' are incompatible.
  //     Type 'string' is not assignable to type 'boolean'.
});
```

Much like explicit type annotations on variables, explicit type arguments may always be specified on a generic function but often aren't necessary. Many TypeScript developers generally only specify them when needed.

The following `logWrapper` usage explicitly specifies `string` both as a type argument and as a function parameter type. Either could be removed:

```
// Type: (input: string) => void
logWrapper<string>((input: string) => { /* ... */ });
```

The `Name<Type>` syntax for specifying a type argument will be the same for other generic constructs throughout this part.

Multiple Function Type Parameters

Functions may define any number of type parameters, separated by commas. Each call of the generic function may resolve its own set of values for each of the type parameters.

In this example, `makeTuple` declares two type parameters and returns a value typed as a read-only tuple with one, then the other:

```
function makeTuple<First, Second>(first: First, second: Second) {  
    return [first, second] as const;  
}  
  
let tuple = makeTuple(true, "abc"); // Type of value: readonly [boolean, string]
```

Note that if a function declares multiple type parameters, calls to that function must explicitly declare either none of the generic types or all of them. TypeScript does not yet support inferring only some of the types of a generic call.

Here, `makePair` also takes in two type parameters, so either neither of them or both of them must be explicitly specified:

```
function makePair<Key, Value>(key: Key, value: Value) {  
    return { key, value };  
}  
  
// Ok: neither type argument provided  
makePair("abc", 123); // Type: { key: string; value: number }  
  
// Ok: both type arguments provided  
makePair<string, number>("abc", 123); // Type: { key: string; value: number }  
makePair<"abc", 123>("abc", 123); // Type: { key: "abc"; value: 123 }  
  
makePair<string>("abc", 123);  
//      ~~~~~  
// Error: Expected 2 type arguments, but got 1.
```

Generic Interfaces

Interfaces may be declared as generic as well. They follow similar generic rules to functions: they may have any number of type parameters declared between a `<` and `>` after

their name. That generic type may later be used elsewhere in their declaration, such as on member types.

The following `Box` declaration has a `T` type parameter for a property. Creating an object declared to be a `Box` with a type argument enforces that the `inside: T` property matches that type argument:

```
interface Box<T> {
    inside: T;
}

let stringyBox: Box<string> = {
    inside: "abc",
};

let numberBox: Box<number> = {
    inside: 123,
}

let incorrectBox: Box<number> = {
    inside: false,
    // Error: Type 'boolean' is not assignable to type 'number'.
}
```

Fun fact: the built-in `Array` methods are defined in TypeScript as a generic interface! `Array` uses a type parameter `T` to represent the type of data stored within an array. Its `pop` and `push` methods look roughly like so:

```
interface Array<T> {
    // ...

    /**
     * Removes the last element from an array and returns it.
     * If the array is empty, undefined is returned and the array is not modified.
     */
    pop(): T | undefined;

    /**
     * Appends new elements to the end of an array,
     * and returns the new length of the array.
     * @param items new elements to add to the array.
     */
    push(...items: T[]): number;
```



```
// ...  
}
```

Inferred Generic Interface Types

As with generic functions, generic interface type arguments may be inferred from usage. TypeScript will do its best to infer type arguments from the types of values provided to a location declared as taking in a generic type.

This `getLast` function declares a type parameter `Value` that is then used for its `node` parameter. TypeScript can then infer `Value` based on the type of whatever value is passed in as an argument. It can even report a type error when an inferred type argument doesn't match the type of a value. Providing `getLast` with an object that doesn't include `next`, or whose inferred `Value` type argument is the same type, is allowed. Mismatching the provided object's `value` and `next.value`, though, is a type error:

```
interface ListNode<Value> {  
    next?: ListNode<Value>;  
    value: Value;  
}  
  
function getLast<Value>(node: ListNode<Value>): Value {  
    return node.next ? getLast(node.next) : node.value;  
}  
  
// Inferred Value type argument: Date  
let lastDate = getLast({  
    value: new Date("09-13-1993"),  
});  
  
// Inferred Value type argument: string  
let lastFruit = getLast({  
    next: {  
        value: "banana",  
    },  
    value: "apple",  
});  
  
// Inferred Value type argument: number  
let lastMismatch = getLast({  
    next: {  
        value: 123  
    },  
});
```



```
    value: false,  
    // ~~~~~  
    // Error: type 'boolean' is not assignable to type 'number'.  
  });
```

Note that if an interface declares type parameters, any type annotations referring to that interface must provide corresponding type arguments. Here, the usage of `CrateLike` is incorrect for not including a type argument:

```
interface CrateLike<T> {  
  contents: T;  
}  
  
let missingGeneric: CrateLike = {  
  // ~~~~~  
  // Error: Generic type 'CrateLike<T>' requires 1 type argument(s).  
  contents: "??"  
};
```

Later in this part, I'll show how to provide default values for type parameters to get around this requirement.

Generic Classes

Classes, like interfaces, can also declare any number of type parameters to be later used on members. Each instance of the class may have a different set of type arguments for its type parameters.

This `Secret` class declares `Key` and `Value` type parameters, then uses them for member properties, constructor parameter types, and a method's parameter and return types:

```
class Secret<Key, Value> {  
  key: Key;  
  value: Value;  
  
  constructor(key: Key, value: Value) {  
    this.key = key;  
    this.value = value;  
  }  
  
  getValue(key: Key): Value | undefined {  
    return this.key === key  
      ? this.value
```



```

        : undefined;
    }
}

const storage = new Secret(12345, "luggage"); // Type: Secret<number, string>

storage.getValue(1987); // Type: string | undefined

```

As with generic interfaces, type annotations using a class must indicate to TypeScript what any generic types on that class are. Later in this part, I'll show how to provide default values for type parameters to get around this requirement for classes too.

Explicit Generic Class Types

Instantiating generic classes goes by the same type arguments inference rules as calling generic functions. If the type argument can be inferred from the type of a parameter to the class constructor, such as the `new Secret(12345, "luggage")` earlier, TypeScript will use the inferred type. Otherwise, if a class type argument can't be inferred from the arguments passed to its constructor, the type argument will default to `unknown`.

This `CurriedCallback` class declares a constructor that takes in a generic function. If the generic function has a known type—such as from an explicit type argument type annotation—then the class instance's `Input` type argument can be informed by it. Otherwise, the class instance's `Input` type argument will default to `unknown`:

```

class CurriedCallback<Input> {
    #callback: (input: Input) => void;

    constructor(callback: (input: Input) => void) {
        this.#callback = (input: Input) => {
            console.log("Input:", input);
            callback(input);
        };
    }

    call(input: Input) {
        this.#callback(input);
    }
}

```

```
// Type: CurriedCallback<string>
new CurriedCallback((input: string) => {
    console.log(input.length);
});

// Type: CurriedCallback<unknown>
new CurriedCallback((input) => {
    console.log(input.length);
    // ~~~~~
    // Error: Property 'length' does not exist on type 'unknown'.
});
```

Class instances may also avoid defaulting to `unknown` by providing explicit type argument(s) the same way other generic function calls do.

Here, `CurriedCallback` from before is now being provided with an explicit `string` for its `Input` type argument, so TypeScript can infer that the callback's `Input` type parameter resolves to `string`:

```
// Type: CurriedCallback<string>
new CurriedCallback<string>((input) => {
    console.log(input.length);
});

new CurriedCallback<string>((input: boolean) => {
    // ~~~~~
    // Argument of type '(input: boolean) => void' is not
    // assignable to parameter of type '(input: string) => void'.
    //   Types of parameters 'input' and 'input' are incompatible.
    //     Type 'string' is not assignable to type 'boolean'.
});
```

Extending Generic Classes

Generic classes can be used as the base class following an `extends` keyword. TypeScript will not attempt to infer type arguments for the base class from usage. Any type arguments without defaults will need to be specified using an explicit type annotation.

The following `SpokenQuote` class provides `string[]` as the `T` type argument for its base class `Quote<T>`:

```

class Quote<T> {
    lines: T;

    constructor(lines: T) {
        this.lines = lines;
    }
}

class SpokenQuote extends Quote<string[]> {
    speak() {
        console.log(this.lines.join("\n"));
    }
}

new Quote("The only real failure is the failure to try.").lines; // Type: string
new Quote([4, 8, 15, 16, 23, 42]).lines; // Type: number[]

new SpokenQuote([
    "Greed is so destructive.",
    "It destroys everything",
]).lines; // Type: string[]

new SpokenQuote([4, 8, 15, 16, 23, 42]);
// ~~~~~
// Error: Argument of type 'number' is not
// assignable to parameter of type 'string'.

```

Generic derived classes can alternately pass their own type argument through to their base class. The type names don't have to match; just for fun, this `AttributedQuote` passes a differently named `Value` type argument to the base class `Quote<T>`:

```

class AttributedQuote<Value> extends Quote<Value> {
    speaker: string

    constructor(value: Value, speaker: string) {
        super(value);
        this.speaker = speaker;
    }
}

// Type: AttributedQuote<string>
// (extending Quote<string>)
new AttributedQuote(
    "The road to success is always under construction.",

```

```
    "Lily Tomlin",  
);
```

Implementing Generic Interfaces

Generic classes may also implement generic interfaces by providing them any necessary type parameters. This works similarly to extending a generic base class: any type parameters on the base interface must be declared by the class.

Here, the `MoviePart` class specifies the `ActingCredit` interface's `Role` type argument as `string`. The `IncorrectExtension` class causes a type complaint because its `role` is type `boolean` despite it providing `string[]` as a type argument to `ActingCredit`:

```
interface ActingCredit<Role> {  
    role: Role;  
}  
  
class MoviePart implements ActingCredit<string> {  
    role: string;  
    speaking: boolean;  
  
    constructor(role: string, speaking: boolean) {  
        this.role = role;  
        this.speaking = speaking;  
    }  
}  
  
const part = new MoviePart("Miranda Priestly", true);  
  
part.role; // Type: string  
  
class IncorrectExtension implements ActingCredit<string> {  
    role: boolean;  
    // ~~~~~  
    // Error: Property 'role' in type 'IncorrectExtension' is not  
    // assignable to the same property in base type 'ActingCredit<string>'.  
    // Type 'boolean' is not assignable to type 'string'.  
}
```

Method Generics

Class methods may declare their own generic types separate from their class instance. Each call to a generic class method

may have a different type argument for each of its type parameters.

This generic `CreatePairFactory` class declares a `Key` type and includes a `createPair` method that also declares a separate `Value` generic type. The return type for `createPair` is then inferred to be `{ key: Key, value: Value }`:

```
class CreatePairFactory<Key> {
  key: Key;

  constructor(key: Key) {
    this.key = key;
  }

  createPair<Value>(value: Value) {
    return { key: this.key, value };
  }
}

// Type: CreatePairFactory<string>
const factory = new CreatePairFactory("role");

// Type: { key: string, value: number }
const numberPair = factory.createPair(10);

// Type: { key: string, value: string }
const stringPair = factory.createPair("Sophie");
```

Static Class Generics

Static members of a class are separate from instance members and aren't associated with any particular instance of the class. They don't have access to any class instances or type information specific to any class instances. As a result, while static class methods can declare their own type parameters, they can't access any type parameters declared on a class.

Here, a `BothLogger` class declares an `OnInstance` type parameter for its `instanceLog` method and a separate `OnStatic` type parameter for its static `staticLog` method. The static method is not able to access the instance `OnInstance` because `OnInstance` is declared for class instances:

```

class BothLogger<OnInstance> {
  instanceLog(value: OnInstance) {
    console.log(value);
    return value;
  }

  static staticLog<OnStatic>(value: OnStatic) {
    let fromInstance: OnInstance;
    // ~~~~~
    // Error: Static members cannot reference class type arguments.

    console.log(value);
    return value;
  }
}

const logger = new BothLogger<number[]>;
logger.instanceLog([1, 2, 3]); // Type: number[]

// Inferred OnStatic type argument: boolean[]
BothLogger.staticLog([false, true]);

// Explicit OnStatic type argument: string
BothLogger.staticLog<string>("You can't change the music of your soul.");

```

Generic Type Aliases

One last construct in TypeScript that can be made generic with type arguments is type aliases. Each type alias may be given any number of type parameters, such as this `Nullish` type receiving a `T`:

```

type Nullish<T> = T | null | undefined;

```

Generic type aliases are commonly used with functions to describe the type of a generic function:

```

type CreatesValue<Input, Output> = (input: Input) => Output;

// Type: (input: string) => number
let creator: CreatesValue<string, number>;

creator = text => text.length; // Ok

creator = text => text.toUpperCase();

```



```
// ~~~~~  
// Error: Type 'string' is not assignable to type 'number'.
```

Generic Discriminated Unions

I mentioned back in that discriminated unions are my favorite feature in all of TypeScript because they beautifully combine a common elegant JavaScript pattern with TypeScript’s type narrowing. My favorite use for discriminated unions is to add a type argument to create a generic “result” type that represents either a successful result with data or a failure with an error.

This `Result` generic type features a `succeeded` discriminant that must be used to narrow a result to whether it’s a success or failure. This means any operation that returns a `Result` can indicate an error or data result, and be assured that consumers will need to check whether the result succeeded:

```
type Result<Data> = FailureResult | SuccessfulResult<Data>;  
  
interface FailureResult {  
  error: Error;  
  succeeded: false;  
}  
  
interface SuccessfulResult<Data> {  
  data: Data;  
  succeeded: true;  
}  
  
function handleResult(result: Result<string>) {  
  if (result.succeeded) {  
    // Type of result: SuccessfulResult<string>  
    console.log(`We did it! ${result.data}`);  
  } else {  
    // Type of result: FailureResult  
    console.error(`Awww... ${result.error}`);  
  }  
  
  result.data;  
  // ~~~~  
  // Error: Property 'data' does not exist on type 'Result<string>'.  
  // Property 'data' does not exist on type 'FailureResult'.  
}
```


Put together, generic types and discriminated types provide a wonderful way to model reusable types like `Result`.

Generic Modifiers

TypeScript includes syntax that allows you to modify the behavior of generic type parameters.

Generic Defaults

I have stated so far that if a generic type is used in a type annotation or as the base of a class `extends` or `implements`, it must provide a type argument for each type parameter. You can get around explicitly providing type arguments by placing an `=` sign followed by a default type after the type parameter's declaration. The default will be used in any subsequent type where the type argument isn't explicitly declared and can't be inferred.

Here, the `Quote` interface takes in a `T` type parameter that defaults to `string` if not provided. The `explicit` variable explicitly sets `T` to `number` while `implicit` and `mismatch` both resolve to `string`:

```
interface Quote<T = string> {
  value: T;
}

let explicit: Quote<number> = { value: 123 };

let implicit: Quote = { value: "Be yourself. The world worships the original." };

let mismatch: Quote = { value: 123 };
//                                     ~~~
// Error: Type 'number' is not assignable to type 'string'.
```

Type parameters can default to earlier type parameters in the same declaration too. Since each type parameter introduces a new type for the declaration, they are available as defaults for later type parameters in that declaration.

This `KeyValuePair` type can have different types for its `Key` and `Value` generics but defaults to keeping them the same—though because `Key` doesn't have a default, it does still need to be inferrable or provided:

```
interface KeyValuePair<Key, Value = Key> {
    key: Key;
    value: Value;
}

// Type: KeyValuePair<string, number>
let allExplicit: KeyValuePair<string, number> = {
    key: "rating",
    value: 10,
};

// Type: KeyValuePair<string, string>
let oneDefaulting: KeyValuePair<string> = {
    key: "rating",
    value: "ten",
};

let firstMissing: KeyValuePair = {
    // ~~~~~
    // Error: Generic type 'KeyValuePair<Key, Value>'
    // requires between 1 and 2 type arguments.
    key: "rating",
    value: 10,
};
```

Keep in mind that all default type parameters must come last in their declaration list, similar to default function parameters. Generic types without a default may not follow generic types with a default.

Here, `inTheEnd` is allowed because all generic types without defaults come before generic types with defaults.

`inTheMiddle` is a problem because a generic type without a default follows types with defaults:

```
function inTheEnd<First, Second, Third = number, Fourth = string>() {} // Ok

function inTheMiddle<First, Second = boolean, Third = number, Fourth>() {}
// ~~~~~
// Error: Required type parameters may not follow optional type parameters.
```

Constrained Generic Types

Generic types by default can be given any type in the world: classes, interfaces, primitives, unions, you name it. However,

some functions are only meant to work with a limited set of types.

TypeScript allows for a type parameter to declare itself as needing to *extend* a type: meaning it's only allowed to alias types that are assignable to that type. The syntax to constrain a type parameter is to place the `extends` keyword after the type parameter's name, followed by a type to constrain it to.

For example, by creating a `WithLength` interface to describe anything that has a `length: number`, we can then allow our generic function to take in any type that has a `length` for its `T` generic. Strings, arrays, and now even objects that just so happen to have a `length: number` are allowed, while type shapes such as `Date` missing that numeric `length` result in a type error:

```
interface WithLength {
  length: number;
}

function logWithLength<T extends WithLength>(input: T) {
  console.log(`Length: ${input.length}`);
  return input;
}

logWithLength("No one can figure out your worth but you."); // Type: string
logWithLength([false, true]); // Type: boolean[]
logWithLength({ length: 123 }); // Type: { length: number }

logWithLength(new Date());
//           ~~~~~
// Error: Argument of type 'Date' is not
// assignable to parameter of type 'WithLength'.
//   Property 'length' is missing in type
//   'Date' but required in type 'WithLength'.
```

I'll cover more type operations you can perform with generics in .

keyof and Constrained Type Parameters

The `keyof` operator introduced in also works well with constrained type parameters. Using `extends` and `keyof` together allows a type parameter to be constrained to the keys

of a previous type parameter. It is also the only way to specify the key of a generic type.

Take this simplified version of the `get` method from the popular library `Lodash`. It takes in a container value, typed as `T`, and a `key` name of one of the keys of `T` to retrieve from `container`. Because the `Key` type parameter is constrained to be a `keyof T`, TypeScript knows this function is allowed to return `T[Key]`:

```
function get<T, Key extends keyof T>(container: T, key: Key) {
    return container[key];
}

const roles = {
    favorite: "Fargo",
    others: ["Almost Famous", "Burn After Reading", "Nomadland"],
};

const favorite = get(roles, "favorite"); // Type: string
const others = get(roles, "others"); // Type: string[]

const missing = get(roles, "extras");
//                               ~~~~~~
// Error: Argument of type '"extras"' is not assignable
// to parameter of type '"favorite" | "others"'.
```

Without `keyof`, there would have been no way to correctly type the generic `key` parameter.

Note the importance of the `Key` type parameter in the previous example. If only `T` is provided as a type parameter, and the `key` parameter is allowed to be any `keyof T`, then the return type will be the union type of all property values in `Container`. This less-specific function declaration doesn't indicate to TypeScript that each call can have a specific `key` via a type argument:

```
function get<T>(container: T, key: keyof T) {
    return container[key];
}

const roles = {
    favorite: "Fargo",
    others: ["Almost Famous", "Burn After Reading", "Nomadland"],
};
```

```
const found = get(roles, "favorite"); // Type: string | string[]
```

Be sure when writing generic functions to know when a parameter’s type depends on a previous parameter’s type. You’ll often need to use constrained type parameters for correct parameter types in those cases.

Promises

Now that you’ve seen how generics work, it’s finally time to talk about a core feature of modern JavaScript that relies on their concepts: Promises! To recap, a Promise in JavaScript represents something that might still be pending, such as a network request. Each Promise provides methods to register callbacks in case the pending action “resolves” (completes successfully) or “rejects” (throws an error).

A Promise’s ability to represent similar actions on any arbitrary value types is a natural fit for TypeScript’s generics. Promises are represented in the TypeScript type system as a `Promise` class with a single type parameter representing the eventual resolved value.

Creating Promises

The `Promise` constructor is typed in TypeScript as taking in a single parameter. That parameter’s type relies on a type parameter declared on the generic `Promise` class. A reduced form would look roughly like this:

```
class PromiseLike<Value> {
  constructor(
    executor: (
      resolve: (value: Value) => void,
      reject: (reason: unknown) => void,
    ) => void,
  ) { /* ... */ }
}
```

Creating a Promise intended to eventually resolve with a value generally necessitates explicitly declaring the type argument of the Promise. TypeScript would default to assuming the parameter type is `unknown` without that explicit

generic type argument. Explicitly providing a type argument to the `Promise` constructor would allow TypeScript to understand the resultant Promise instance's resolved type:

```
// Type: Promise<unknown>
const resolvesUnknown = new Promise((resolve) => {
  setTimeout(() => resolve("Done!"), 1000);
});

// Type: Promise<string>
const resolvesString = new Promise<string>((resolve) => {
  setTimeout(() => resolve("Done!"), 1000);
});
```

A Promise's generic `.then` method introduces a new type parameter representing the resolved value of the Promise it returns.

For example, the following code creates a `textEventually` Promise that resolves with a `string` value after a second, as well as a `lengthEventually` that immediately resolves with a `number`:

```
// Type: Promise<string>
const textEventually = new Promise<string>((resolve) => {
  setTimeout(() => resolve("Done!"), 1000);
});

// Type: Promise<number>
const lengthEventually = textEventually.then((text) => text.length)
```

Async Functions

Any function declared in JavaScript with the `async` keyword returns a `Promise`. If a value returned by an `async` function in JavaScript isn't a Thenable (an object with a `.then()` method; in practice almost always a Promise), it will be wrapped in a `Promise` as if `Promise.resolve` was called on it. TypeScript recognizes this and will infer the return type of an `async` function to always be a `Promise` for whatever value is returned.

Here, `lengthAfterSecond` returns a `Promise<number>` directly, while `lengthImmediately` is inferred to return a

`Promise<number>` because it is `async` and directly returns a `number`:

```
// Type: (text: string) => Promise<number>
async function lengthAfterSecond(text: string) {
    await new Promise((resolve) => setTimeout(resolve, 1000))
    return text.length;
}

// Type: (text: string) => Promise<number>
async function lengthImmediately(text: string) {
    return text.length;
}
```

Any manually declared return type on an `async` function therefore must always be a `Promise` type, even if the function doesn't explicitly mention Promises in its implementation:

```
// Ok
async function givesPromiseForString(): Promise<string> {
    return "Done!";
}

async function givesString(): string {
    // ~~~~~~
    // Error: The return type of an async function
    // or method must be the global Promise<T> type.
    return "Done!";
}
```

Using Generics Right

As in the `Promise<Value>` implementations earlier in this part, although generics can give us a lot of flexibility in describing types in code, they can become rather complex quite quickly. Programmers new to TypeScript often go through a phase of overusing generics to the point of making code confusing to read and overly complex to work with. TypeScript best practice is generally to use generics only when necessary, and to be clear about what they're used for when they are.

WARNING

Most code you write in TypeScript should not heavily use generics to the point of confusion. However, types for utility libraries, particularly general-use modules, may sometimes need to heavily use them. Understanding generics is particularly useful to be able to work effectively with those utility types.

The Golden Rule of Generics

One quick test that can help show whether a type parameter is necessary for a function is it should be used at least twice. Generics describe relationships between types, so if a generic type parameter only appears in one place, it can't possibly be defining a relationship between multiple types.

Each function type parameter should be used for a parameter and then also for at least one other parameter and/or the return type of the function.

For example, this `logInput` function uses its `Input` type parameter exactly once, to declare its `input` parameter:

```
function logInput<Input extends string>(input: Input) {  
    console.log("Hi!", input);  
}
```

Unlike the `identify` functions earlier in the part, `logInput` doesn't do anything with its type parameter such as returning or declaring more parameters. There is therefore not much use to declaring that `Input` type parameter. We can rewrite `logInput` without it:

```
function logInput(input: string) {  
    console.log("Hi!", input);  
}
```

Effective TypeScript by Dan Vanderkam (O'Reilly, 2019) contains several excellent tips for how to work with generics, including a section titled “The Golden Rule of Generics.” I highly recommend reading *Effective TypeScript* and that section especially if you're finding yourself spending a lot of time wrestling with generics in your code.

Generic Naming Conventions

The standard naming convention for type parameters in many languages, TypeScript included, is to default to calling a first type argument “T” (for “type” or “template”) and if subsequent type parameters exist, calling them “U,” “V,” and so on.

If some contextual information is known about how the type argument is supposed to be used, the convention sometimes extends to using the first letter of the term for that usage: for example, state management libraries might refer to a generic state as “S.” “K” and “V” often refer to keys and values in data structures.

Unfortunately, naming a type argument with one letter can be just as confusing as naming a function or variable with just one character:

```
// What on earth are L and V?!  
function labelBox<L, V>(l: L, v: V) { /* ... */ }
```

When the intent of a generic isn’t clear from a single-letter T, it’s best to use descriptive generic type names that indicate what the type is used for:

```
// Much more clear.  
function labelBox<Label, Value>(label: Label, value: Value) { /* ... */ }
```

Whenever a construct has multiple type parameters, or the purpose of a single type argument isn’t immediately clear, consider using fully written names for readability instead of single-letter abbreviations.

In this part, you made classes, functions, interfaces, and type aliases “generic” by allowing them to work with type parameters:

- Using type parameters to represent types different between uses of a construct
- Providing explicit or implicit type arguments when calling generic functions
- Using generic interfaces to represent generic object types

- Adding type parameters to classes, and how that impacts their types
- Adding type parameters to type aliases, in particular with discriminated type unions
- Modifying generic type parameters with defaults (`=`) and constraints (`extends`)
- How Promises and `async` functions use generics to represent asynchronous data flow
- Best practices with generics, including their Golden Rule and naming conventions

Thus concludes the *Features* section of this material.

Congratulations: you now know all the most important syntax and type-checking features in the TypeScript type system for most projects!